



Banco de Dados

Prof. Elson de Abreu



Stored Procedure e Function

Stored Procedure

- Um procedimento armazenado (Stored Procedure) é um conjunto de instruções implementadas com a linguagem Transact-SQL. Fica armazenado no SGBD de forma pré-compilada, aguardando que um usuário solicite sua execução. Vantagens em se utilizar:
 - Modularidade: um procedimento pode ser dividido em várias partes e tais partes armazenadas no servidor, permitindo que uma alteração nas regras seja refletida por todo o sistema.
 - Redução de I/O: com a passagem de parâmetros há uma redução no tráfego de dados pela rede, uma vez que os dados a serem alterados já estão no próprio servidor.
 - Rapidez na execução: como já estão pré-compilados, as ações também já estão pré-carregadas, dependendo somente dos valores dos parâmetros.
- *Obs.: Não iremos tratar sobre códigos implementados diretamente em CLR.*

Stored Procedure

- O tamanho máximo do bloco de comandos Transact-SQL deve ser de 128 MB.
- Podem fazer referência à VIEWS, TRIGGERS e também à tabelas temporárias.
- Os comandos CREATE DEFAULT, CREATE RULE, CREATE TRIGGER e CREATE VIEW não podem constar em uma construção de Stored Procedure.
- Caso crie uma tabela temporária local, essa tabela somente existirá até o fim da execução do procedimento.

Stored Procedure

- Sintaxe:

```
CREATE PROC [ EDURE ] procedure_name [ ; number ]  
    [ { @parameter data_type }  
      [ = default ] [ OUTPUT ] ] [ ,...n ]  
  
    [ WITH  
      {ENCRYPTION} ]  
  
    AS sql_statement [ ...n ]
```

Obs.: Para executar a procedure utilize: **EXEC**[UTE] *procedure_name* { @*parameter* }

Exemplo 1 *Sem Parâmetros*

```
-- UTILIZANDO O BD "EMPRESA"
```

```
USE EMPRESA
```

```
GO
```

```
-- RETORNA OS EMPREGADOS DO DEPARTAMENTO DE CÓDIGO IGUAL A 20
```

```
-- ESTA PROCEDURE NÃO POSSUI PARÂMETROS
```

```
CREATE PROCEDURE EMP20
```

```
AS
```

```
    SELECT * FROM EMP WHERE DEPTNO = 20
```

```
GO
```

```
-- EXECUTANDO A PROCEDURE
```

```
EXEC EMP20
```

```
GO
```

Exemplo 2 *Com Parâmetros*

```
CREATE PROCEDURE REGISTRA_ALUGUEL_FITA @CODFITA INT,  
    @CODCLIENTE INT, @DATA DATETIME
```

```
AS
```

```
-- INSERINDO REGISTRO DE LOCAÇÃO
```

```
INSERT INTO LOCACAO (CODFITA, CODCLIENTE, DATA)
```

```
VALUES (@CODFITA, @CODCLIENTE, @DATA)
```

```
-- ATUALIZANDO O STATUS DA FITA
```

```
UPDATE FITA
```

```
    SET STATUS = 'L'
```

```
WHERE CODFITA = @CODFITA
```

```
GO
```

```
-- EXECUTANDO A PROCEDURE – OBS: NÃO UTILIZE A FUNÇÃO GETDATE()  
    DIRETAMENTE NA CHAMADA
```

```
DECLARE @DATAATUAL DATETIME
```

```
    SET @DATAATUAL = GETDATE()
```

```
EXEC REGISTRA_ALUGUEL_FITA 260, 100, @DATAATUAL
```

```
GO
```

Ou

```
DECLARE @DATAATUAL DATETIME
```

```
    SET @DATAATUAL = GETDATE()
```

```
EXEC REGISTRA_ALUGUEL_FITA @CODFITA = 260, @CODCLIENTE = 100,  
    @DATA = @DATAATUAL
```

```
GO
```

Stored Procedure

- Alterando uma stored procedure:

```
ALTER PROCEDURE EMP20  
AS  
SELECT * FROM emp  
WHERE deptno = 20 and salario > 1000
```

Obs.: não afeta as stored procedures aninhadas.

- Excluindo uma stored procedure:

```
DROP PROCEDURE EMP20
```


Function

- É uma rotina armazenada, similar a uma stored procedure, que retorna um valor.
- As funções podem ter no máximo 1024 parâmetros.
- A diferença básica entre “SP” e function é que esta obrigatoriamente deve retornar um valor. Existem outras diferenças, porém é recomendável a leitura sobre os tipos de functions para obter maiores detalhes sobre tais diferenças.

Function

- Sintaxe:

```
CREATE FUNCTION [ owner_name. ] function_name
    ( [ { @parameter_name [AS] scalar_parameter_data_type [ = default ] } [ ,...n ] ] )
RETURNS @return_variable TABLE < table_type_definition >
[ WITH < function_option > [ [,] ...n ] ]
[ AS ]
BEGIN
    function_body
    RETURN
END
```

- A sintaxe acima refere-se a uma função escalar. É recomendável consultar o manual online da ferramenta para saber sobre os outros tipos de funções.

Exemplo

```
CREATE FUNCTION CIDADE_CLIENTE (@CODCLIENTE INT)
    RETURNS VARCHAR(100)
AS
BEGIN
    DECLARE @CIDADE VARCHAR(30)

    SELECT @CIDADE = CIDADE + '/' + ESTADO
    FROM CLIENTE
    WHERE CODCLIENTE = @CODCLIENTE

    IF @CIDADE IS NULL
    BEGIN
        SET @CIDADE = 'SEM VALOR'
    END

    RETURN @CIDADE
END
GO
```

```
DECLARE @CIDADE VARCHAR(100)
SET @CIDADE = DBO.CIDADE_CLIENTE(100)
PRINT @CIDADE
GO
```

Ou

```
SELECT DBO.CIDADE_CLIENTE(100)
GO
```

Cursors T-SQL

- Cursor é um objeto que aponta para um conjunto de linhas retornadas por uma instrução SELECT. Tal conjunto de linhas pode ser manipulado linha por linha.
- Cursores podem ser definidos em qualquer bloco de Transact-SQL (procedures, functions, triggers, ...).

- Comandos básicos:

DECLARE	declara um cursor
OPEN	abre um cursor
FETCH	busca uma linha
CLOSE	fecha um cursor
DEALLOCATE	remove a referência a um cursor

- *Recomenda-se a leitura sobre os diferentes tipos de cursor: Static (padrão), Keyset, Dynamic e Firehose.*

Cursors T-SQL

- Sintaxe (declaração – padrão SQL 92):

```
DECLARE cursor_name  
    [ INSENSITIVE ] [ SCROLL ] CURSOR  
FOR select_statement
```

- Sintaxe (fetch):

```
FETCH  
    [ [ NEXT | PRIOR | FIRST | LAST  
        | ABSOLUTE { n | @nvar }  
        | RELATIVE { n | @nvar }  
    ]  
    FROM  
    ]  
{ [ GLOBAL ] cursor_name }  
[ INTO @variable_name [ ,...n ] ]
```

As instruções de posicionamento do cursor são:

- **FETCH FIRST** avança para o primeiro registo do cursor.
- **FETCH NEXT** avança para o registo seguinte.
- **FETCH PRIOR** avança para o registo anterior.
- **FETCH LAST** avança para o último registo do cursor.
- **FETCH ABSOLUTE n** avança para posição n do cursor.
- **FETCH RELATIVE n** avança n registos a partir do registo atual do cursor.

Para utilizar os comandos de fetch (com exceção do NEXT) é necessário declarar um cursor do tipo SCROLL.

Cursores T-SQL

- Duas importantes funções para controlar um cursor são:
 - @@cursor_rows: exibe a quantidade de linhas do cursor
 - @@fetch_status: exibe o status do último comando FETCH

0 : bem sucedido

-1 : a busca falhou ou fez com que o cursor fosse além do conjunto de resultados

- Exemplo:

```
while @@fetch_status = 0
```

```
... bloco_comandos
```

Exemplo

```
DECLARE @empno int, @nome varchar(50), @i int
```

```
set @i = 0
```

```
DECLARE crsr_emp CURSOR
```

```
FOR select empno, nome from emp where deptno = 10 order by nome desc
```

```
OPEN crsr_emp -- é necessário abrir o cursor antes de utiliza-lo
```

```
Print 'O cursor possui ' + convert(varchar, @@cursor_rows) + ' linhas '
```

```
Print "
```

```
fetch next from crsr_emp -- carregando as variáveis com os valores da primeira  
linha do cursor
```

```
into @empno, @nome
```

```
while @@fetch_status = 0
```

```
begin
```

```
set @i = @i + 1
```

```
print 'Estamos na posicao ' + convert(varchar, @i) + ' do cursor'
```

```
print 'Empno ' + convert(varchar, @empno) + ', nome ' + convert(varchar,  
@nome)
```

```
Print " -- imprimindo uma linha em branco (espaçamento)
```

```
fetch next from crsr_emp
```

```
into @empno, @nome
```

```
end
```

```
close crsr_emp -- após utilizar o cursor, é necessário fecha-lo e liberar a memória  
com deallocate
```

```
deallocate crsr_emp
```

Atividade 1 *Stored Procedure*

- Modifique a “SP” REGISTRA_ALUGUEL_FITA aplicando os conceitos de tratamento de erros e transações explícitas.

A fita só poderá ser locada se o status estiver como “disponível”.

- Crie uma “SP” chamada REGISTRA_ENTREGA_FITA. Determine seus parâmetros, faça tratamento de erros e controle de transação. A entrega da fita só poderá ser registrada caso seu status estiver como “locada” e existir um registro de locação para o cliente indicado com a data de devolução não preenchida.

- *Utilizar o bd “LOCADORA”.*

Atividade 2 *Stored Procedure*

- Criar uma “SP” que receba como parâmetros o código, nome e local de um departamento. Se já existir um departamento com este código, atualize o departamento com o nome e local passados como parâmetro. Caso contrário, insira este departamento na tabela DEPT.
- *Utilizar o bd “EMPRESA”.*

Atividade 3 *Cursor*

- Crie uma “SP” chamada ATUALIZA_STATUS que atualize o campo STATUS de todas as fitas da tabela FITA. Esta atualização deve ser realizada baseando-se na tabela de locações de fitas. Para as fitas alugadas e que ainda não foram devolvidas, atualize o seu STATUS com o valor 'L'. Para as fitas que já foram devolvidas, atualize o seu STATUS com o valor 'D'. A finalidade desta “SP” é corrigir as divergências entre a tabela LOCACAO e a tabela FITA.
- *Utilizar o bd “LOCADORA”.*

Atividade 4 *Function*

- Crie uma function chamada “QUANT_LOCACOES” que receba como parâmetro o código do cliente e retorne a quantidade de locações feitas por ele.
- Crie uma query que exiba os dados abaixo e utilize a função para exibir a coluna “Locacoes”.

CODCLIENTE	NOME	LOCACOES
100	João da Silva	4
110	Maria José de Souza	2
120	Antônio Carlos Ferreira	1
130	Patrícia dos Santos	1
140	Paulo Correia	1
150	Daniela Marinho	3
160	Fabírcia Passos	2
170	Danilo Passos	1
180	Roberto Assunção	1
190	Sílvia Cordeiro	1
200	Ricardo Andrade	0

- *Utilizar o bd “LOCADORA”.*